

CS714: Secure Computation

Adithya Vadapalli

IIT Kanpur

January 20, 2026

Definition

A probability space is a triple $(S, \mathcal{F}, \text{Pr})$ where:

- S is the **sample space**
- $\mathcal{F}$ is the **event space**
- Pr is a probability measure

Definition

A probability space is a triple $(S, \mathcal{F}, \text{Pr})$ where:

- S is the **sample space**
- $\mathcal{F}$ is the **event space**
- Pr is a probability measure

Example

- $S = \{1, 2, 3, 4, 5, 6\}$ (fair die)
- $\mathcal{F} = 2^S$
- $\text{Pr}[\{i\}] = \frac{1}{6}$

Key Fact

If S is countable, then for any event $E \subseteq S$:

$$\Pr[E] = \sum_{e \in E} \Pr[e]$$

Example

Let $E = \{2, 4, 6\}$.

$$\Pr[E] = \frac{1}{6} + \frac{1}{6} + \frac{1}{6} = \frac{1}{2}$$

Random Variables

Definition

A random variable is a function:

$$X : S \rightarrow A$$

where A is called the **range**.

Example

Let:

$$S = \{1, 2, 3, 4, 5, 6\}$$

Define:

$$X(s) = s \bmod 2$$

- $X = 0$ for even outcomes
- $X = 1$ for odd outcomes

Events of the Form $X = x$

Definition

The event $X = x$ means:

$$X^{-1}(x) = \{s \in S \mid X(s) = x\}$$

Example

What is $X^{-1}(0)$? (Here X^{-1} is pre-image!)

Events of the Form $X = x$

Definition

The event $X = x$ means:

$$X^{-1}(x) = \{s \in S \mid X(s) = x\}$$

Example

What is $X^{-1}(0)$? (Here X^{-1} is pre-image!)

Example

$$X^{-1}(0) = \{2, 4, 6\}$$

$$\Pr[X = 0] = \Pr[\{2, 4, 6\}] = \frac{1}{2}$$

Events of the Form $X \in C$

Definition

For $C \subseteq A$: $X \in C := \bigcup_{x \in C} X^{-1}(x)$

Example

Let: $C = \{0\}$

Example

$$X \in C = \{2, 4, 6\}$$

$$\Pr[X \in C] = \Pr[X = 0] = \frac{1}{2}$$

Support of a Random Variable

Definition

The support of X is:

$$\text{supp}(X) = \{x \mid \Pr[X = x] > 0\}$$

Example

For the die parity variable:

$$\text{supp}(X) = \{0, 1\}$$

Predicate Random Variables

Definition

A predicate random variable has range $\{\perp, \top\}$.

We write:

$$\Pr[P] := \Pr[P = \top]$$

What is $\Pr[P]$?

$$\text{Let: } P(s) = \begin{cases} \top & \text{if } s \in \{1, 2\} \\ \perp & \text{otherwise} \end{cases}$$

Example

$$\Pr[P] = \frac{2}{6} = \frac{1}{3}$$

Combining Random Variables

Definition

If:

$$X : S \rightarrow A, \quad Y : S \rightarrow B$$

and

$$g : A \times B \rightarrow C$$

then:

$$g(X, Y)(s) = g(X(s), Y(s))$$

Example

Two dice:

$$Z = X + Y$$

where Z is the sum.

Combining Random Variables

Basic Idea

- Given random variables $X : S \rightarrow A$, $Y : S \rightarrow B$
- Define a new random variable $Z = g(X, Y)$ for any deterministic function $g : A \times B \rightarrow C$.

Interpretation

Combining random variables: *applying a function pointwise on outcomes in the sample space.*

Combining via Arithmetic Operations

Example: Sum

Let

$$Z = X + Y$$

where $X, Y : S \rightarrow \mathbb{Z}_p$. Then

$$Z(s) = X(s) + Y(s) \pmod{p}.$$

Distribution

$$\Pr[Z = z] = \sum_{x+y=z} \Pr[X = x, Y = y]$$

Joint Random Variables

Joint Variable

The pair (X, Y) is itself a random variable:

$$(X, Y) : S \rightarrow A \times B$$

defined by

$$(X, Y)(s) = (X(s), Y(s)).$$

Joint Distribution

$$\Pr[X = x, Y = y] = \Pr(X^{-1}(x) \cap Y^{-1}(y))$$

Example: Joint Random Variables

Question

Two fair coins are tossed.

Let

X = outcome of the first toss, Y = outcome of the second toss.

What is the joint distribution of (X, Y) ?

Answer

The joint random variable (X, Y) takes values in $\{H, T\} \times \{H, T\}$.

$$\Pr[X = H, Y = H] = \Pr[X = H, Y = T] = \Pr[X = T, Y = H] = \Pr[X = T, Y = T] = \frac{1}{4}.$$

Assignment Notation

Important

The notation:

$$Y \leftarrow Z$$

means **definition**, not randomness.

Example

If:

$$Y \leftarrow 2 \cdot X^2$$

then:

$$\Pr[X^2 = Y/2] = 1$$

Statistical Distance

Definition

Let X_0 and X_1 be random variables with the same range D . The **statistical distance** between X_0 and X_1 is:

$$\Delta(X_0, X_1) = \frac{1}{2} \sum_{d \in D} |\Pr[X_0 = d] - \Pr[X_1 = d]|$$

Upper Bound

What is the upper bound of $\Delta(X_0, X_1)$?

Statistical Distance: Upper Bound

For any $d \in D$,

$$|\Pr[X_0 = d] - \Pr[X_1 = d]| \leq \Pr[X_0 = d] + \Pr[X_1 = d].$$

Summing over $d \in D$:

$$\sum_{d \in D} |\Pr[X_0 = d] - \Pr[X_1 = d]| \leq 2.$$

Hence:

$$0 \leq \Delta(X_0, X_1) \leq 1.$$

When Is Distance Equal to 1?

$$\Delta(X_0, X_1) = 1 \iff \text{Supports of } X_0 \text{ and } X_1 \text{ are disjoint.}$$

That is:

$$\forall d \in D, \Pr[X_0 = d] \cdot \Pr[X_1 = d] = 0.$$

One sample distinguishes with certainty.

Example: Three-Face Die

Let $D = \{1, 2, 3\}$.

Define:

$$\Pr[X_0 = 1] = 1, \quad \Pr[X_1 = 3] = 1.$$

Then:

$$\Delta(X_0, X_1) = \frac{1}{2}(|1 - 0| + |0 - 0| + |0 - 1|) = 1.$$

Maximum possible statistical distance.

Intuition Behind Statistical Distance

Intuition

Statistical distance measures how well one can distinguish two distributions by observing a single sample.

- $\Delta = 0 \Rightarrow$ distributions are identical
- $\Delta = 1 \Rightarrow$ distributions are completely disjoint

Cryptographic Meaning

Maximum distinguishing advantage of any adversary is exactly $\Delta(X_0, X_1)$.

Example 1: Identical Distributions

Setup

Let:

$$X_0 \sim \text{Uniform}(\{0, 1\}), \quad X_1 \sim \text{Uniform}(\{0, 1\})$$

Probabilities

$$\Pr[X_0 = 0] = \Pr[X_1 = 0] = \frac{1}{2}$$

$$\Pr[X_0 = 1] = \Pr[X_1 = 1] = \frac{1}{2}$$

Distance

$$\Delta(X_0, X_1) = 0$$

Example 2: Slightly Different Distributions

Setup

Let $D = \{0, 1\}$ and:

$$\Pr[X_0 = 0] = \frac{1}{2}, \quad \Pr[X_0 = 1] = \frac{1}{2}$$

$$\Pr[X_1 = 0] = \frac{3}{4}, \quad \Pr[X_1 = 1] = \frac{1}{4}$$

Distance Computation

$$\Delta(X_0, X_1) = \frac{1}{2} \left(\left| \frac{1}{2} - \frac{3}{4} \right| + \left| \frac{1}{2} - \frac{1}{4} \right| \right) = \frac{1}{4}$$

Example 3: Disjoint Distributions

Setup

Let:

$$X_0 = 0 \text{ always, } X_1 = 1 \text{ always}$$

Probabilities

$$\Pr[X_0 = 0] = 1, \quad \Pr[X_1 = 1] = 1$$

Distance

$$\Delta(X_0, X_1) = 1$$

Interpretation

Perfect distinguishability.

Equivalent Definition (Using Events)

Theorem

$$\Delta(X_0, X_1) = \max_{C \subseteq D} |\Pr[X_0 \in C] - \Pr[X_1 \in C]|$$

Meaning

The statistical distance equals the maximum probability gap over all events of the form $X \in C$.

Example Using $X \in C$

Setup

From earlier example:

$$\Pr[X_0 = 0] = \frac{1}{2}, \quad \Pr[X_1 = 0] = \frac{3}{4}$$

Choice of Event

Choose:

$$C = \{0\}$$

Distinguishing Advantage

$$|\Pr[X_0 \in C] - \Pr[X_1 \in C]| = \left| \frac{1}{2} - \frac{3}{4} \right| = \frac{1}{4}$$

Statistical Distance and Predicates

Key Connection

For any predicate P :

$$|\Pr[P(X_0)] - \Pr[P(X_1)]| \leq \Delta(X_0, X_1)$$

Cryptography View

No test (predicate) can distinguish X_0 and X_1 with advantage greater than their stat distance.

One-Time Pad (OTP)

Setting

- Message space: $\{0, 1\}^n$
- Key space: $\{0, 1\}^n$
- Key $K \leftarrow U_n$ (used once)

Encryption

$$\text{Enc}_K(M) = M \oplus K$$

Decryption

$$\text{Dec}_K(C) = C \oplus K$$

The adversary observes:

$$C = M \oplus K$$

where:

- M is unknown
- K is uniformly random

Question

Does C reveal anything about M ?

Induced Ciphertext Distribution

Fix any message $m \in \{0, 1\}^n$.

Let:

$$C_m := m \oplus U_n$$

Since XOR with a fixed string is a permutation:

$$C_m \sim U_n$$

Thus, for every message m :

$$\Pr[C_m = c] = 2^{-n} \quad \forall c$$

Two Messages, Same Distribution

Let $m_0, m_1 \in \{0, 1\}^n$ be any two messages.

Then:

$$C_{m_0} = m_0 \oplus U_n \quad C_{m_1} = m_1 \oplus U_n$$

Both are uniformly distributed over $\{0, 1\}^n$.

Statistical Indistinguishability

For any (even unbounded) distinguisher $\mathcal{D}$:

$$\Pr[\mathcal{D}(C_{m_0}) = 1] = \Pr[\mathcal{D}(C_{m_1}) = 1]$$

Therefore:

$$\text{Adv}_{\mathcal{D}}(C_{m_0}, C_{m_1}) = 0$$

The statistical distance is:

$$\Delta(C_{m_0}, C_{m_1}) = 0$$

Perfect Secrecy

- Ciphertexts for different messages are **identically distributed**
- The adversary gains **zero information**

Probabilistic Model

- Cryptographic algorithms are inherently randomized
 - Security is defined over *distributions*, not single runs
 - We need a formal model to reason about randomness and outputs
-
- Algorithm A is a deterministic TM with an extra random tape
 - Random tape: $r \in \{0, 1\}^\infty$
 - Inputs and randomness are separated

Algorithm Execution

For inputs $x_1, \dots, x_\ell$ and randomness r : $A(x_1, \dots, x_\ell; r)$ denotes the output of A on random tape r .

Fix inputs $x_1, \dots, x_\ell$.

- Sample $r \leftarrow \{0, 1\}^\infty$
- $A(x_1, \dots, x_\ell)$ is the induced random variable

For any output y : $\Pr[A(x_1, \dots, x_\ell) = y] = \Pr_r[A(x_1, \dots, x_\ell; r) = y]$

Termination Convention

- Algorithms terminate with probability 1
- Non-termination is mapped to a special symbol $\perp$
- Ensures well-defined random variables

Distinguishing Advantage

Distinguishing between Random Variables

For an algorithm D and two random variables X_0 and X_1 , we need a measure of the ability of D to distinguish between X_0 and X_1

The Game

- 1 Sample a uniformly random bit $b \leftarrow \{0, 1\}$
- 2 Let $x \leftarrow X_b$
- 3 $c \leftarrow D(x)$

c is D 's guess at b . Notice that if the guess is fixed at $c = 0$, then the algorithm, D will be correct with probability $1/2$.

Defining the Advantage

$$\begin{aligned}\Pr[c = b] &= \frac{1}{2} \Pr[c = b \mid b = 0] + \frac{1}{2} \Pr[c = b \mid b = 1] \\&= \frac{1}{2} \Pr[c = 0 \mid b = 0] + \frac{1}{2} \Pr[c = 1 \mid b = 1] \\&= \frac{1}{2} \Pr[D(X_0) = 0] + \frac{1}{2} \Pr[D(X_1) = 1] \\&= \frac{1}{2} \left(\Pr[D(X_0) = 0] + 1 - \Pr[D(X_1) = 0] \right) \\&= \frac{1}{2} + \frac{1}{2} \left(\Pr[D(X_0) = 0] - \Pr[D(X_1) = 0] \right)\end{aligned}$$

Main Equation

$$2 \left| \Pr[c = b] - \frac{1}{2} \right| = \left| \Pr[D(X_0) = 0] - \Pr[D(X_1) = 0] \right|$$

Why this quantity is called the Advantage

Advantage Definition

$$\text{Adv} := 2 \left| \Pr[c = b] - \frac{1}{2} \right|$$

Intuition

- A random guess succeeds with probability $\frac{1}{2}$.
- $\Pr[c = b] - \frac{1}{2}$ measures excess success over guessing.
- Absolute value allows flipping a consistently wrong distinguisher.
- Factor 2 normalizes the value to lie in $[0, 1]$.

Thus, advantage = “how much better than random guessing.”

Homework

Prove that: $\text{Adv}_{\mathcal{D}}(X_0, X_1) = \delta(D(X_0), D(X_1))$

Statistical Indistinguishability

Negligible Function

A function $\delta: \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is called negligible if for every $c \in \mathbb{N}$, there exists $\kappa_c \in \mathbb{N}$ such that

$$\delta(\kappa) \leq \kappa^{-c} \quad \text{for all } \kappa \geq \kappa_c.$$

Intuition

Negligible means *vanishes faster than any inverse polynomial*. Such probabilities are considered effectively zero for polynomial-time adversaries.

Examples and Non-Examples

- **Negligible:** $2^{-\kappa}$, $2^{-\sqrt{\kappa}}$
- **Not negligible:** $1/\kappa$, $1/\kappa^2$

Formal Definition

Let X_0 and X_1 be random variables with the same range.

- X_0 and X_1 are *statistically indistinguishable*, written

$$X_0 \equiv^{\text{stat}} X_1,$$

if $\delta(X_0(\kappa), X_1(\kappa))$ is negligible in κ .

- X_0 and X_1 are *perfectly indistinguishable*, written

$$X_0 \equiv^{\text{perf}} X_1,$$

if $\delta(X_0(\kappa), X_1(\kappa)) = 0$ for all κ .

- Perfect indistinguishability means the output distributions are identical.
- Statistical indistinguishability means they are extremely close: differing only by a negligible amount of probability mass.
- Statistically indistinguishable distributions can be viewed as “essentially impossible to distinguish.”

Basic Properties

Let $\mathcal{A}$ be any algorithm, and let X_0, X_1, X_2 be families of random variables.

The following hold for $\equiv^{\text{stat}}$:

- ① $X_0 \equiv^{\text{stat}} X_0$
- ② If $X_0 \equiv^{\text{stat}} X_1$, then $X_1 \equiv^{\text{stat}} X_0$
- ③ If $X_0 \equiv^{\text{stat}} X_1$ and $X_1 \equiv^{\text{stat}} X_2$, then $X_0 \equiv^{\text{stat}} X_2$
- ④ If $X_0 \equiv^{\text{stat}} X_1$, then $\mathcal{A}(X_0) \equiv^{\text{stat}} \mathcal{A}(X_1)$

Perfect Indistinguishability

The same properties also hold when $\equiv^{\text{stat}}$ is replaced by $\equiv^{\text{perf}}$.

Security as a Guessing Game

We model an attacker as an algorithm that:

- sees some data,
- tries to guess how that data was generated.

Security as a Guessing Game

We model an attacker as an algorithm that:

- sees some data,
- tries to guess how that data was generated.

Security means:

No efficient algorithm can guess significantly better than chance.

IND-CPA = Distinguishability

- Adversary chooses (m_0, m_1)
- Challenger encrypts m_b
- Adversary guesses b

IND-CPA = Distinguishability

- Adversary chooses (m_0, m_1)
- Challenger encrypts m_b
- Adversary guesses b

Advantage

$$\text{Adv}^{\text{IND-CPA}} = \left| \Pr[b' = b] - \frac{1}{2} \right|$$

IND-CPA = Distinguishability

- Adversary chooses (m_0, m_1)
- Challenger encrypts m_b
- Adversary guesses b

Advantage

$$\text{Adv}^{\text{IND-CPA}} = \left| \Pr[b' = b] - \frac{1}{2} \right|$$

Security Meaning

Encryption is IND-CPA secure iff no PPT distinguisher has non-negligible advantage.

Indistinguishability-Based Encryption Security

Let $(\text{KeyGen}, \text{Enc}, \text{Dec})$ be a public-key encryption scheme.

Security is defined via a *game* between:

- a challenger, and
- an adversary trying to distinguish encryptions.

We compare two threat models:

IND-CPA vs. **IND-CCA.**

IND-CPA Game

- 1 Challenger runs $(pk, sk) \leftarrow \text{KeyGen}$ and gives pk to $\mathcal{A}$.
- 2 $\mathcal{A}$ may query an *encryption oracle* $\text{Enc}(pk, \cdot)$.
- 3 $\mathcal{A}$ submits two messages (m_0, m_1) of equal length.
- 4 Challenger samples $b \leftarrow \{0, 1\}$ and gives

$$c^* = \text{Enc}(pk, m_b).$$

- 5 $\mathcal{A}$ outputs a guess b' .

$$\text{Adv}_{\mathcal{A}}^{\text{IND-CPA}} = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

IND-CCA Game

Same as IND-CPA, except:

- $\mathcal{A}$ also has access to a *decryption oracle*

$\text{Dec}(sk, \cdot),$

- with the restriction that it cannot query $\text{Dec}(sk, c^*)$.

$$\text{Adv}_{\mathcal{A}}^{\text{IND-CCA}} = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

What IND-CCA Captures

Intuition

IND-CCA models *active attackers* who can tamper with ciphertexts and observe how they decrypt.

- Strong notion for real-world security
- Protects against malleability and padding attacks
- Required for secure protocols and APIs

Example: Encrypt-then-MAC, CCA-secure KEMs.

$$\text{IND-CCA} \implies \text{IND-CPA}$$

- IND-CPA is necessary but not sufficient
- IND-CCA is the gold standard for encryption

Key Takeaway

IND-CPA hides *plaintexts*. IND-CCA hides plaintexts *even under active attacks*.

Public-Key Cryptography (Idea)

Public-key cryptography uses *two keys*:

- a **public key** for encryption,
- a **secret key** for decryption.

A public-key cryptosystem consists of three algorithms:

$$(G, E, D)$$

- G : key generation,
- E : encryption using the public key,
- D : decryption using the secret key.

Example: ElGamal Encryption (Overview)

Let $\mathbb{G}$ be a cyclic group of prime order q with generator g .

Key generation:

- Choose $x \leftarrow \mathbb{Z}_q$
- Public key: $pk = g^x$
- Secret key: $sk = x$

Encryption of message $m \in \mathbb{G}$:

- Choose $r \leftarrow \mathbb{Z}_q$
- Output ciphertext:

$$(c_1, c_2) = (g^r, m \cdot pk^r)$$

Decryption:

$$m = c_2 / c_1^x$$

Claim. ElGamal is IND-CPA secure assuming the *Decisional Diffie–Hellman (DDH)* assumption holds in $\mathbb{G}$.

DDH Assumption

The DDH assumption states that the distributions

$$(g, g^a, g^b, g^{ab}) \quad \text{and} \quad (g, g^a, g^b, g^c)$$

are computationally indistinguishable, where

$$a, b, c \leftarrow \mathbb{Z}_q.$$

IND-CPA Proof Sketch (Hybrid Argument)

Let (m_0, m_1) be the adversary's challenge messages.

Real experiment:

$$(g^r, m_b \cdot g^{xr})$$

Hybrid experiment: Replace g^{xr} with a random group element g^c :

$$(g^r, m_b \cdot g^c)$$

By DDH, the adversary cannot distinguish these two experiments.

Why the Hybrid Is Secure

In the hybrid experiment:

$$(g^r, m_b \cdot g^c)$$

- g^c is uniform in $\mathbb{G}$
- $m_b \cdot g^c$ is independent of b

Thus, the ciphertext reveals no information about b .

Conclusion:

ElGamal $\equiv^{\text{IND-CPA}}$ secure under DDH.

ElGamal is NOT IND-CCA Secure

Claim. ElGamal encryption is *not* IND-CCA secure.

Why ElGamal Fails IND-CCA

ElGamal is *malleable*.

Given a ciphertext

$$(c_1, c_2) = (g^r, m \cdot g^{xr}),$$

anyone can construct:

$$(c_1, c_2 \cdot h)$$

which decrypts to:

$$m \cdot h.$$

- Adversary receives challenge ciphertext

$$(c_1, c_2) = E_{pk}(m_b).$$

- It submits $(c_1, c_2 \cdot g)$ to the decryption oracle.
- Oracle returns $m_b \cdot g$.
- Adversary recovers m_b and learns b .

This query is allowed since the ciphertext is different from the challenge.

Two Competing Distributions

Let:

- X_0 and X_1 be random variables
- Both range over the same domain $\mathcal{X}$

Examples:

- output of a cryptographic algorithm
- output of a truly random process

Two Competing Distributions

Let:

- X_0 and X_1 be random variables
- Both range over the same domain $\mathcal{X}$

Examples:

- output of a cryptographic algorithm
- output of a truly random process

Question:

Can an algorithm tell which one it is seeing?

Key Takeaways

- Security is defined via **distinguishing experiments**
- Advantage measures how well an attack works
- “Secure” means *negligible advantage for all efficient algorithms*

Pseudorandom Generators

A function

$$G : \{0, 1\}^n \rightarrow \{0, 1\}^{\ell(n)}, \quad \ell(n) > n$$

is a *pseudorandom generator (PRG)* if its output is computationally indistinguishable from uniform.

Real vs Random Experiments

Real World

$s \leftarrow \{0, 1\}^n$, output $G(s)$ (Pseudorandom)

Ideal World

$r \leftarrow \{0, 1\}^{\ell(n)}$, output r (True Randomness)

$\text{Real}_G(n)$ vs $\text{Rand}(n)$

Distinguishing Algorithm for PRGs

Recall

- A distinguisher D is a PPT algorithm
- Input: a string $y \in \{0, 1\}^{\ell(n)}$
- Output: a bit in $\{0, 1\}$

$$\text{Adv}_G^{\text{prg}}(D, n) = |\Pr[D(G(s)) = 1] - \Pr[D(r) = 1]|$$

where:

- $s \leftarrow \{0, 1\}^n$
- $r \leftarrow \{0, 1\}^{\ell(n)}$

PRG Security Definition

Definition

G is a PRG if for every PPT distinguisher D ,

$$\text{Adv}_G^{\text{prg}}(D, n) \leq \text{negl}(n)$$

Interpretation

- Distinguisher formalizes *any feasible statistical test*
- If no PPT D can distinguish, $G(s)$ “looks random”
- This is computational indistinguishability

Families of Random Variables

Family of random variables. A family of random variables is a sequence

$$X = \{X(\kappa)\}_{\kappa \in \mathbb{N}},$$

where for each security parameter κ , $X(\kappa)$ is a random variable (possibly over a different probability space).

Let $\mathcal{A}$ be a two-input algorithm that takes κ as its first input. We define

$$\mathcal{A}(X) \stackrel{\text{def}}{=} \{\mathcal{A}(\kappa, X(\kappa))\}_{\kappa \in \mathbb{N}}.$$

That is, for each κ , we sample $x \leftarrow X(\kappa)$ and output $\mathcal{A}(\kappa, x)$.

Random Variables vs Families of Random Variables

Random variable (standard view). A random variable is a function

$$X : \Omega \rightarrow \mathcal{D},$$

from a sample space Ω (with a probability distribution) to a domain $\mathcal{D}$.

Cryptographic setting. Objects depend on a security parameter κ , so we consider a *family* of random variables

$$\{X(\kappa)\}_{\kappa \in \mathbb{N}}, \quad X(\kappa) : \Omega_{\kappa} \rightarrow \mathcal{D}_{\kappa}.$$

Each $X(\kappa)$ is an ordinary random variable; the family captures how the distribution evolves as κ grows.

Distinguishing Advantage of a Class of Algorithms

In cryptography, we are typically not interested if *one particular algorithm* can distinguish a class of random variables.

Let $\mathcal{A}$ be a class of distinguishers.

We say X_0 and X_1 are *indistinguishable by $\mathcal{A}$* if

$$\mathcal{A}(X_0) \equiv^{\text{stat}} \mathcal{A}(X_1) \quad \text{for all } \mathcal{A} \in \mathcal{A}.$$

$$X_0 \equiv^{\mathcal{A}} X_1$$

Indistinguishability by a Class of Algorithms

Definition. Let

$$X_0 = \{X_0(\kappa)\}_{\kappa \in \mathbb{N}} \quad \text{and} \quad X_1 = \{X_1(\kappa)\}_{\kappa \in \mathbb{N}}$$

be families of random variables, and let $\mathcal{A}$ be a class of distinguishers.

We say that X_0 and X_1 are *indistinguishable by $\mathcal{A}$* if

$$\mathcal{A}(X_0) \equiv^{\text{stat}} \mathcal{A}(X_1) \quad \text{for all } A \in \mathcal{A}.$$

We write

$$X_0 \equiv^{\mathcal{A}} X_1.$$

Computational Indistinguishability

If $\mathcal{A}$ is the class of all polynomial-time algorithms and

$$X_0 \equiv^{\mathcal{A}} X_1,$$

then we say that X_0 and X_1 are *computationally indistinguishable*, and write

$$X_0 \equiv^{\text{comp}} X_1.$$

If they are not computationally indistinguishable, we write

$$X_0 \not\equiv^{\text{comp}} X_1.$$

Public-Key Encryption Setup

Let (G, E, D) be a public-key encryption scheme.

- On input κ , the key generator outputs

$$(pk, sk) \leftarrow G(\kappa).$$

- On input (pk, m) , the encryption algorithm outputs

$$C \leftarrow E_{pk}(m),$$

using internal randomness.

- On input (sk, C) , the decryption algorithm outputs

$$m \leftarrow D_{sk}(C),$$

with correctness $D_{sk}(E_{pk}(m)) = m$.

Defining the Random Variable Families

For $b \in \{0, 1\}$, define a family of random variables

$$X_b = \{X_b(\kappa)\}_{\kappa \in \mathbb{N}}$$

as follows:

- 1 Sample $(pk, sk) \leftarrow G(\kappa)$.
- 2 Sample $C \leftarrow E_{pk}(b)$.
- 3 Output $X_b(\kappa) = (pk, C)$.

Intuitively: $X_b(\kappa)$ outputs a public key together with an encryption of the bit b .

Statistical Distance Between X_0 and X_1

- A ciphertext C cannot be both an encryption of 0 and 1.
- Therefore, the supports of $X_0(\kappa)$ and $X_1(\kappa)$ are disjoint.
- Hence,

$$\delta(X_0(\kappa), X_1(\kappa)) = 1.$$

Thus,

$$X_0 \not\equiv^{\text{stat}} X_1 \quad \text{and} \quad X_0 \not\equiv^{\text{perf}} X_1.$$

- For public-key encryption, the standard security notion is *semantic security*.
- Semantic security implies that no polynomial-time adversary can distinguish an encryption of 0 from an encryption of 1.
- Equivalently, the distinguishing advantage of any PPT algorithm is negligible in κ .

Therefore,

$$X_0 \equiv^{\text{comp}} X_1.$$

Computational vs. Statistical Indistinguishability

Give example of two families of random variables that are perfectly distinguishable statistically, however are computationally indistinguishable.

Distinguishing Advantage

Let $\mathcal{A}$ be a (possibly randomized) algorithm and let

$$X_0 = \{X_0(\kappa)\}_{\kappa \in \mathbb{N}}, \quad X_1 = \{X_1(\kappa)\}_{\kappa \in \mathbb{N}}$$

be families of random variables.

Definition of Advantage

For each security parameter κ ,

$$\text{Adv}_{\mathcal{A}}(X_0, X_1)(\kappa) \stackrel{\text{def}}{=} \text{Adv}_{\mathcal{A}(\kappa, \cdot)}(X_0(\kappa), X_1(\kappa)),$$

where $\mathcal{A}(\kappa, \cdot)$ denotes $\mathcal{A}$ with κ fixed and one sample as input.

Why OTP Is Stronger Than PRGs

- OTP achieves **statistical** (perfect) indistinguishability
- Security holds against **all** adversaries
- No computational assumptions required

In contrast:

- PRGs achieve only computational indistinguishability

Indistinguishability: Unified View

Two distributions D_0, D_1 over the same domain.

A distinguisher A outputs 1 if it believes the sample came from D_0 .

$$\text{Adv}_A(D_0, D_1) = |\Pr[A(D_0) = 1] - \Pr[A(D_1) = 1]|$$

Perfect indistinguishability:

$$\Pr[D_0 = x] = \Pr[D_1 = x] \quad \forall x$$

$\Rightarrow \text{Adv}_A = 0$ for all (unbounded) A

Example: One-Time Pad

Ciphertext distributions are identical.

Statistical indistinguishability:

$$\Delta(D_0, D_1) \leq \text{negl}(n)$$

Allows tiny, information-theoretically negligible leakage.

Statistical Example: Truncated PRG

Let $G : \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$ be a PRG.

- D_0 : first n bits of $G(s)$, $s \leftarrow \{0, 1\}^n$
- D_1 : uniform over $\{0, 1\}^n$

For each $y \in \{0, 1\}^n$:

$$\Pr[D_0 = y] = 2^{-n} \pm \text{negl}(n)$$

$\Rightarrow$ Statistical but not perfect

Why Full PRG Is Not Statistical

$$|\text{Supp}(D_0)| \leq 2^n, \quad |\text{Supp}(D_1)| = 2^{2n}$$

For $x \notin \text{Im}(G)$:

$$\Pr[D_0 = x] = 0, \quad \Pr[D_1 = x] = 2^{-2n}$$

$$\Delta(D_0, D_1) \geq \frac{1}{2}(1 - 2^{-n}) \approx 1$$

PRGs are only computationally secure.

Computational Indistinguishability

If $\mathcal{A}$ is the class of all polynomial-time algorithms and

$$X_0 \equiv^{\mathcal{A}} X_1,$$

then we say that X_0 and X_1 are *computationally indistinguishable*, and write

$$X_0 \equiv^{\text{comp}} X_1.$$

If they are not computationally indistinguishable, we write

$$X_0 \not\equiv^{\text{comp}} X_1.$$

Hierarchy and Takeaway

Perfect $\subsetneq$ Statistical $\subsetneq$ Computational

- Perfect: distributions identical
- Statistical: negligible information leakage
- Computational: leakage exists but is efficiently hidden

Interactive Agents and Ports

Interactive Agent

- An *interactive agent* has:
 - a set of **input ports** $\text{In}(A)$
 - a set of **output ports** $\text{Out}(A)$
- Ports are identified by *names*
- Agents interact by connecting outputs to inputs with the same name

Port Compatibility

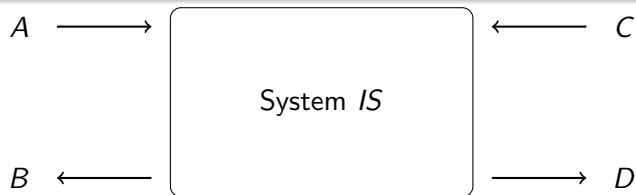
Interactive agents $A_1, \dots, A_n$ are *port compatible* if for all $i \neq j$:

$$\text{In}(A_i) \cap \text{In}(A_j) = \emptyset \quad \text{and} \quad \text{Out}(A_i) \cap \text{Out}(A_j) = \emptyset$$

- No two agents have the same input port name
- No two agents have the same output port name

Informal Definition

It is a computational device that *sends* and *receives* messages on named ports.



Interactive Agents

Interactive Agent is tuple: $A = (\text{In}, \text{Out}, \text{State}, \text{Msg}, T, \sigma_0)$.

- 1 In is a finite set of *inports*
- 2 Out is a finite set of *outports*
- 3 State is a set of possible *states*
- 4 Msg is a set of *messages*
- 5 T is a transition algorithm.
- 6 σ_0 is the current state.

Transitions

Input to Transition Algorithm is (κ, σ, I)

- 1 κ is the security parameter.
- 2 $\sigma \in \text{State}$
- 3 $I \in \text{In}, I \in \text{Msg}, I \in \text{EOQ}, \text{SNT}, \text{ILM}$

EOQ: End of Queue SNT: Sent Message Notation ILM: Illegal Message

Output of Transition Algorithm

The output of transition algorithm can be:

- 1 (send, P, m)
- 2 (read, P)
- 3 (return, P)

The purpose of the return port is to specify which port to activate next in the larger system.

Activation of an Agent

- Running an agent is called an *activation*.
- In one activation, the agent may:
 - read from ports multiple times,
 - send on ports multiple times,
 - update its internal state.
- The initial state is σ_0 .
- At the end of the activation, the agent specifies a *return port*.

Algorithm: Activate Agent (Inputs)

Inputs to the Activate Agent

- Security parameter κ
- Current state σ
- Activation port $AP \in \text{In}(A)$
- A queue Q_P for each port $P \in \text{Ports}(A)$

Initialization

- 1 Let $I \leftarrow AP$.
- 2 Compute $(\sigma', c) \leftarrow T(\kappa, \sigma, I)$.
- 3 Update the state: $\sigma \leftarrow \sigma'$.

Algorithm: Processing Commands

Process the command c as follows:

Send

- **send:** if $c = (\text{send}, P, m)$
 - Append m to the queue Q_P
 - Set $I \leftarrow \text{SNT}$
 - Go to the next step

Read

- **read:** if $c = (\text{read}, P)$
 - If Q_P is empty, set $I \leftarrow \text{EOQ}$
 - Otherwise:
 - Remove the front element m from Q_P
 - Set $I \leftarrow m$
 - If $I \notin \text{Msg}(A)$, set $I \leftarrow \text{ILM}$
 - Go to the next step

Algorithm: Return

Return

- **return:** if $c = (\text{return}, RP)$
 - The activation terminates
 - Output the new state σ and return port RP
 - Queues $\{Q_P\}_{P \in \text{Ports}(A)}$ may have been updated

Shorthand for Describing Agents

- “Agent sends the activation token on P ” means:

$\text{return}(P)$

- “Agent sends m on P ” means:
 - send m on P , then
 - return on P
- “On input m on P , do $A(m)$ ”:
 - read from P
 - if $m \notin \{\text{EOQ}, \text{ILM}\}$, execute $A(m)$

Definition: Responsive Agents

Responsive

An agent is *responsive* if for all contexts $(\kappa, \sigma, AP, \{Q_P\})$ the activation returns with probability 1.

Polyresponsive

An agent is *polyresponsive* if it returns after executing at most a polynomial (in κ) number of commands.

Stepwise Polytime

An agent is stepwise polytime if:

- $\text{Msg} = \{0, 1\}^{p(\kappa)}$ for polynomial p
- T can be computed in expected polynomial time

Definition: Polytime Agent

Polytime

An agent is *polytime* if it is both:

- stepwise polytime, and
- polyresponsive.

Port Compatibility and Composition

Port Compatibility

Interactive agents $A_1, \dots, A_n$ are *port compatible* if for all $i \neq j$:

$$\text{In}(A_i) \cap \text{In}(A_j) = \emptyset \quad \text{and} \quad \text{Out}(A_i) \cap \text{Out}(A_j) = \emptyset.$$

Interactive Systems

If $A_1, \dots, A_n$ are port compatible, then $IS = \{A_1, \dots, A_n\}$ is called an *interactive system*.

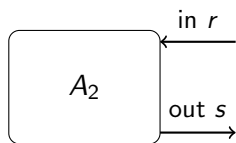
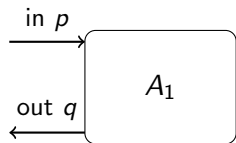
Composition

Two interactive systems IS_1, IS_2 are port compatible if all their agents are port compatible. In this case, define:

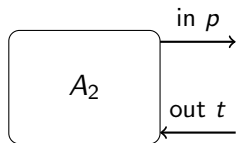
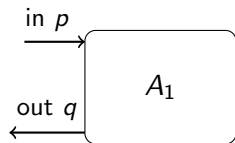
$$IS_1 \diamond IS_2 \stackrel{\text{def}}{=} IS_1 \cup IS_2.$$

Otherwise, define $IS_1 \diamond IS_2 = \perp$.

Port Compatibility: Visual Illustration



✓ Port compatible



× Same input port name

Left: disjoint input/output names $\Rightarrow$ can form an interactive system.

Right: shared input name $\Rightarrow IS_1 \diamond IS_2 = \perp$.

Example: Port Compatible Agents

Example

Let:

$$\text{In}(A_1) = \{x\}, \quad \text{Out}(A_1) = \{y\}$$

$$\text{In}(A_2) = \{y\}, \quad \text{Out}(A_2) = \{z\}$$

- Input ports are disjoint: $\{x\} \cap \{y\} = \emptyset$
- Output ports are disjoint: $\{y\} \cap \{z\} = \emptyset$
- Hence, A_1 and A_2 are **port compatible**

Definition

If agents $A_1, \dots, A_n$ are port compatible, then

$$IS = \{A_1, \dots, A_n\}$$

is an *interactive system*.

- A system is simply a collection of compatible agents
- No execution semantics yet — only structure

System Composition

Two interactive systems IS_1 and IS_2 are *port compatible* if all agents together are port compatible.

- If compatible:

$$IS_1 \diamond IS_2 = IS_1 \cup IS_2$$

- Otherwise:

$$IS_1 \diamond IS_2 = \perp$$

Open Input Ports of a System

Definition

For an interactive system IS , define:

$$\text{In}(IS) \stackrel{\text{def}}{=} \left(\bigcup_{A \in IS} \text{In}(A) \right) \setminus \left(\bigcup_{A \in IS} \text{Out}(A) \right)$$

- Inputs not connected to any output
- Called the *open input ports* of IS

Open Output Ports of a System

Definition

For an interactive system IS , define:

$$\text{Out}(IS) \stackrel{\text{def}}{=} \left(\bigcup_{A \in IS} \text{Out}(A) \right) \setminus \left(\bigcup_{A \in IS} \text{In}(A) \right)$$

- Outputs not connected to any input
- Called the *open output ports* of IS

Open and Closed Interactive Systems

- $\text{In}(IS)$ and $\text{Out}(IS)$ are the *open ports* of IS
- IS is called *closed* if:

$$\text{In}(IS) = \emptyset \quad \text{and} \quad \text{Out}(IS) = \emptyset$$

- A closed interactive system IS is *executable* if:
 - there exists an agent $A \in IS$
 - A has an input port named ε
 - A has an output port named ε
- Purpose:
 - well-defined initial activation port
 - well-defined final return port

Activation-Driven Execution

- Execution is *activation driven*
- In each step, exactly one agent is activated
- Initially:
 - activate agent A_ε
 - activation port is ε

Passing the Activation Token

- After an activation:
 - the agent specifies a return port
 - the next agent activated is the one with that input port
- Execution continues by passing an *activation token*
- Termination occurs when an agent returns on port ε

Algorithm: Execute System

Inputs and Outputs

- Input:
 - Interactive system IS
 - Security parameter κ
- Output:
 - Message m or ∞

Initialization

- 1 Initialize agent states:

$$\forall A \in IS, \quad \sigma_A \leftarrow \sigma_0(A)$$

- 2 Initialize queues:

$$\forall P \in \bigcup_{A \in IS} \text{Ports}(A), \quad Q_P \leftarrow \varepsilon$$

Algorithm: Execute System (Control Flow)

Initial Activation

- ③ Set initial activation port:

$$AP \leftarrow \varepsilon$$

- ④ Let A_{AP} be the agent with input port named AP

Agent Activation

- ⑤ Activate agent A_{AP} on:

- security parameter κ
- current state $\sigma_{A_{AP}}$
- activation port AP
- queues $\{Q_P\}_{P \in \text{Ports}(A_{AP})}$

- ⑥ Receive:

- updated state σ'
- return port RP

Algorithm: Execute System (Control Flow)

Termination Check

- ⑧ If $RP \neq \varepsilon$, go back to agent selection
- ⑨ If $RP = \varepsilon$, terminate with output m :

$$m = \begin{cases} 0 & \text{if } Q_\varepsilon \text{ is empty} \\ \text{front}(Q_\varepsilon) & \text{otherwise} \end{cases}$$

Update

- ⑥ Update state:

$$\sigma_{AAP} \leftarrow \sigma'$$

- ⑦ Update activation port:

$$AP \leftarrow RP$$

Execute System: Summary

- Single-agent activation at a time
- Control passed via return ports
- Execution starts and ends on ε
- Output determined by queue Q_ε

Execution of an Interactive System

Executable Interactive System

An interactive system IS is called *executable* if:

- IS is *closed*, and
- there exists an agent $A \in IS$ such that

$$\varepsilon \in \text{In}(A) \quad \text{and} \quad \varepsilon \in \text{Out}(A).$$

Definition 2.15: Output of Execution

- Let IS be executable and let $\kappa \in \mathbb{N}$ be the security parameter.
- Execute IS using `ALGORITHM EXECUTE SYSTEM`.
- If execution terminates with output m :

$$IS(\kappa) := m.$$

- If execution does not terminate:

$$IS(\kappa) := \infty.$$

Remark

$IS(\kappa)$ is a random variable with range $\text{Msg} \cup \{\infty\}$.

Execution as a Random Variable

Definition

For an executable interactive system IS and security parameter k :

$$IS(k) = \text{output of Execute System}$$

- If execution does not halt: $IS(k) = \infty$
- Hence $IS(k)$ is a **random variable**
- Often write $\{IS(k)\}_{k \in \mathbb{N}}$

Example 2.16: Uniform Sampling via Rejection

- Let A be an interactive agent with:

$$\text{In}(A) = \{\varepsilon, 1\}, \quad \text{Out}(A) = \{\varepsilon, 1\}.$$

- On input on ε or 1:
 - sample $r \leftarrow \{0, 1\}^\ell$, where $\ell = \lceil \log_2 \kappa \rceil$
 - interpret r as $R \in \mathbb{Z}_{2^\ell}$

Example 2.17: Two-Agent Interactive System

- Agent A_1 :

$$\text{In}(A_1) = \{\varepsilon, 1\}, \quad \text{Out}(A_1) = \{\varepsilon, 2\}.$$

- On input on ε :
 - sample $c \leftarrow \{0, 1\}$
 - send c on port 2

Example 2.17: Agent Interaction

- Agent A_1 on input $e \in \{0, 1\}$ on port 1:

- send e on ε

- Agent A_2 :

$$\text{In}(A_2) = \{2\}, \quad \text{Out}(A_2) = \{1\}.$$

- On input c on port 2:

- sample $d \leftarrow \{0, 1\}$
- send $c + d$ on port 1

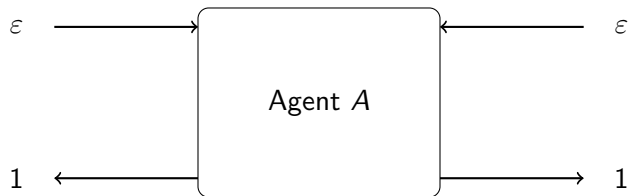
Example 2.17: Output Distribution

Let $IS = \{A_1, A_2\}$.

Distribution of $IS(\kappa)$

$$\Pr[IS(\kappa) = 0] = \frac{1}{4}, \quad \Pr[IS(\kappa) = 1] = \frac{1}{2}, \quad \Pr[IS(\kappa) = \infty] = \frac{1}{4}.$$

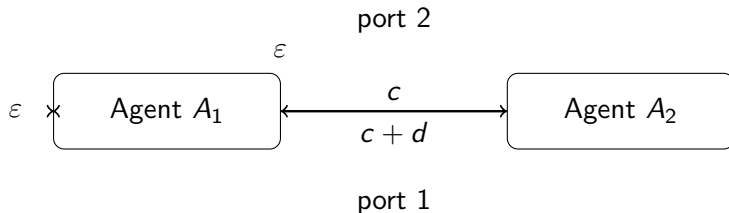
Execution Diagram: Example 2.16



Sample $R \in \mathbb{Z}_{2^\ell}$
If $R < \kappa$: return on ε
Else: return on 1

- Port 1 creates a self-loop (retry)
- Returning on ε terminates execution

Execution Diagram: Example 2.17



A_1 : sample $c \in \{0, 1\}$, send on 2

A_2 : sample $d \in \{0, 1\}$, send $c + d$ on 1

A_1 : output on ε

- Execution may terminate or loop
- Nonzero probability of non-termination

Definition 2.18: Closure

Closure of an Interactive System

Let IS be an interactive system. A *closure* of IS is an interactive system Z such that:

- $IS \circ Z \neq \perp$, and
- $IS \circ Z$ is executable.

Definition 2.18: Port Compatibility

- Z closes IS by matching open ports:

$$\text{In}(Z) = \text{Out}(IS), \quad \text{Out}(Z) = \text{In}(IS).$$

- All outputs of IS are connected to inputs of Z
- All outputs of Z are connected to inputs of IS

- The port name ε is *reserved for closures*.
- Normal interactive systems do not use ε internally.
- The closure Z defines:
 - initial activation, and
 - output of the execution.
- Ensures:
 - clean start of execution
 - well-defined output

Role of the Environment

- We model the observer as an **environment** $\mathcal{Z}$
- $\mathcal{Z}$:
 - connects to the open ports of the system
 - interacts arbitrarily with it
 - outputs a **single bit** $c \in \{0, 1\}$
- The bit c is $\mathcal{Z}$'s guess of which system it is interacting with

Closing the System

- The environment $\mathcal{Z}$ acts as a *closure*
- Both compositions must be executable:

$$IS_0 \diamond \mathcal{Z} \quad \text{and} \quad IS_1 \diamond \mathcal{Z}$$

- Their executions produce output bits

$$(IS_0 \diamond \mathcal{Z})(k), (IS_1 \diamond \mathcal{Z})(k)$$

Why Closures Matter

- Environment / adversary modeled as closure
- Closure defines:
 - inputs to the system
 - when execution halts
 - final output
- Essential for:
 - simulation-based security
 - UC-style definitions

Big Picture

- Interactive systems = modular protocol components
- Execution defines induced random variables
- Closures model environments and adversaries
- Foundation for modern cryptographic security definitions

Indistinguishable Interactive Systems

- Goal: formalize when two interactive systems

IS_0 and IS_1

are *behaviorally indistinguishable*

- Intuition:
 - An external observer interacts with the system
 - The observer should not be able to tell which system it is connected to

Mapping Interactive Systems to MPC

- Clean abstraction for:
 - Interactive protocols
 - Cryptographic reductions
 - MPC protocol execution
 - Composition behaves algebraically
 - Separates *structure* from *execution*
-
- **Agents** $\leftrightarrow$ MPC machines
 - **Ports** $\leftrightarrow$ communication channels
 - **Composition** $\leftrightarrow$ protocol execution
 - **Closed system** $\leftrightarrow$ standalone MPC execution

This model captures MPC *without* fixing a network or scheduler

Example 1: Two-Party MPC Protocol

Agents

- P_1 : Party holding input x_1
 - P_2 : Party holding input x_2
-
- $\text{In}(P_1) = \{\epsilon, m_{21}\}$
 - $\text{Out}(P_1) = \{m_{12}\}$
 - $\text{In}(P_2) = \{m_{12}\}$
 - $\text{Out}(P_2) = \{m_{21}\}$

Ports as MPC Messages

- Each message type has a unique port name
- Example:
 - m_{12} : message from P_1 to P_2
 - m_{21} : message from P_2 to P_1
- Port compatibility enforces:
 - no ambiguous receivers
 - no message collisions

Interactive System

$$IS_{\text{MPC}} = \{P_1, P_2\}$$

- Composition connects m_{12} and m_{21} internally
- Remaining open port: ϵ
- Execution begins at ϵ

- If all message ports are internally connected:

$$\text{In}(IS) = \text{Out}(IS) = \emptyset$$

- The protocol runs without external interaction
- Models:
 - honest-but-curious execution
 - fixed input delivery

Example 2: MPC with an Environment

Agents

- $\mathcal{Z}$: Environment
 - P_1, P_2 : MPC parties
-
- $\mathcal{Z}$ provides inputs and receives outputs
 - Modeled via open ports

Open Ports = Environment Interface

- Inputs:

$$\text{Out}(\mathcal{Z}) = \{x_1, x_2\}$$

- Outputs:

$$\text{In}(\mathcal{Z}) = \{y_1, y_2\}$$

- These ports remain open in IS

Exactly matches the UC notion of environment interaction

Example 3: Ideal Functionality

Agent $\mathcal{F}_f$

- $\text{In}(\mathcal{F}_f) = \{x_1, x_2\}$
- $\text{Out}(\mathcal{F}_f) = \{y_1, y_2\}$
- $\mathcal{F}_f$ computes $f(x_1, x_2)$
- Sends outputs to corresponding parties

- **Real system:**

$$IS_{\text{real}} = \{\mathcal{Z}, P_1, P_2\}$$

- **Ideal system:**

$$IS_{\text{ideal}} = \{\mathcal{Z}, \mathcal{F}_f\}$$

- Security = indistinguishability of executions

Execution as Token Passing

- Activation starts at ϵ
- Environment activates a party
- Party outputs a message port
- Next agent with matching input is activated

Captures synchronous and asynchronous MPC uniformly

Why This Matters for MPC Proofs

- Formalizes:
 - message delivery
 - scheduling
 - environment control
- Avoids informal “round-based” reasoning
- Scales cleanly to:
 - multi-party protocols
 - hybrid functionalities
 - protocol composition

Perfect Indistinguishability

Definition

We say IS_0 and IS_1 are **perfectly indistinguishable**, written

$$IS_0 \equiv_{\text{perf}} IS_1$$

if for all environments $\mathcal{Z}$:

$$\{(IS_0 \diamond \mathcal{Z})(k)\}_{k \in \mathbb{N}} \equiv \{(IS_1 \diamond \mathcal{Z})(k)\}_{k \in \mathbb{N}}$$

The output distributions are *identical*

Example: Perfect Indistinguishability

Example

IS_0 and IS_1 differ only in:

- internal variable names
- unreachable code paths

- No environment can observe the difference
- Hence:

$$IS_0 \equiv_{\text{perf}} IS_1$$

Statistical Indistinguishability

Definition

We say IS_0 and IS_1 are **statistically indistinguishable**, written

$$IS_0 \equiv_{\text{stat}} IS_1$$

if for all environments $\mathcal{Z}$ making polynomially many activations:

$$\{(IS_0 \diamond \mathcal{Z})(k)\}_{k \in \mathbb{N}} \approx \{(IS_1 \diamond \mathcal{Z})(k)\}_{k \in \mathbb{N}}$$

Distance between distributions is negligible

Example: Statistical but not Perfect

Example

IS_0 and IS_1 behave identically except:

- with probability 2^{-k}
- they output different bits

- Perfect indistinguishability fails
- Statistical indistinguishability holds

Indistinguishability w.r.t. a Class of Environments

Definition

Let Env be a class of environments. We say:

$$IS_0 \equiv_{\text{Env}} IS_1$$

if indistinguishability holds for all $\mathcal{Z} \in \text{Env}$.

- Restricts the power of the distinguisher
- Useful for modular security definitions

Definition

We say $IS_0 \equiv_{\text{comp}} IS_1$ if:

$$IS_0 \equiv_{\text{Env}} IS_1$$

- where Env is the class of
- polynomial-time interactive systems

This is the standard cryptographic notion

Public-Key Cryptosystems

A public-key cryptosystem consists of three algorithms

$$(G, E, D),$$

where:

- G is the key-generation algorithm,
- E is the encryption algorithm,
- D is the decryption algorithm.

Security is defined by comparing two *interactive systems*.

The IND Interactive Experiment

For $b \in \{0, 1\}$, define an interactive agent A_b .

Input ports:

$$\text{In}(A_b) = \{\text{keygen}, \text{msgs}, \text{dec}\}.$$

Output ports:

$$\text{Out}(A_b) = \{\text{pk}, \text{target}, \text{plaintext}\}.$$

Behavior of the Key Generation Port

On the first message to keygen:

- Sample $(pk, sk) \leftarrow G(\kappa)$.
- Output pk on port pk .

All subsequent messages on keygen are ignored and do not change the internal state.

Behavior of the Challenge Message Port

On the first message (m_0, m_1) on `msgs`, where $|m_0| = |m_1|$:

- Sample the challenge ciphertext

$$c^* \leftarrow E_{pk}(m_b).$$

- Output c^* on port `target`.

All subsequent messages on `msgs` are ignored.

Behavior of the Decryption Port

On input ciphertext c on port dec:

- Output $m \leftarrow D_{sk}(c)$ on port plaintext.
- If c^* has been defined, inputs with $c = c^*$ are ignored.

Interpretation of IND-CCA

IND-CCA strengthens IND-CPA by allowing:

- Access to a decryption oracle,
- Except on the challenge ciphertext c^* .

The task remains hard even with decryption access.

Existence of Secure Schemes

Cryptosystems satisfying IND-CPA and IND-CCA security can be constructed under standard assumptions, such as:

- the RSA assumption,
- the discrete logarithm assumption.